

Rust 过程宏

从零开始的探索之旅

Typst 编译 — 2026-07-15

这不是一本 API 手册。

这是一个开发者发现过程宏的全程记录——

从第一个重复的代码开始，到最后一个自定义 `derive` 结束。

目录

1	序章：重复代码的问题	5
1.1	方案一：复制粘贴	5
1.2	方案二：macro_rules!	5
1.3	方案三：过程宏	6
1.4	本教程的目标	6
2	从 macro_rules! 到过程宏	7
2.1	两种宏的对比	7
2.2	项目结构	7
2.3	第一个过程宏	7
2.4	三种过程宏	8
2.5	函数式宏示例	8
2.6	macro_rules! 做不到的事	9
2.7	过程宏的代价	9
2.8	小结	9
3	TokenStream — 编译器眼中的代码	10
3.1	TokenStream 的组成	10
3.2	打印 TokenStream	10
3.3	Group: 括号里的世界	11
3.4	Ident、Punct、Literal	11
3.5	TokenStream 是可解析的	11
3.6	字符串与 TokenStream 的互相转换	12
3.7	小结	12
4	解析：用 syn 把 TokenStream 变成数据结构	13
4.1	第一条解析代码	13
4.2	DeriveInput	13
4.3	解析不同的数据类型	14
4.4	不只是 DeriveInput	14
4.5	属性解析：读取 #[...]	15
4.6	Parse trait: 自定义解析	15
4.7	syn 的 features	16
4.8	动手验证	16
4.9	小结	16
5	生成代码：quote! 魔法	17
5.1	第一个 quote! 示例	17
5.2	插值方式	18
5.3	为结构体生成 getter 方法	18
5.4	条件生成	18
5.5	proc_macro2 与 proc_macro	19
5.6	ToTokens trait	19
5.7	常见陷阱	19
5.8	小结	19
6	派生宏 — 自动实现 Trait	21
6.1	从一个简单的 trait 开始	21
6.2	派生宏的签名	21
6.3	处理泛型	22
6.4	访问字段	22
6.5	带属性的派生宏	23
6.6	完整的模式	24
6.7	Derive 宏的设计模式	24

6.8	小结	24
7	属性宏 — 函数的注解	25
7.1	属性宏的结构	25
7.2	示例: <code>log_call</code>	25
7.3	用 <code>attr</code> 参数做配置	26
7.4	典型用例	26
7.5	属性宏 vs <code>Derive</code> 宏	27
7.6	常见陷阱	27
7.7	小结	27
8	函数式宏 — 自定义 DSL	28
8.1	函数式宏的签名	28
8.2	案例: 简化的 <code>json!</code> 宏	28
8.3	生成代码	29
8.4	解析自定义 DSL	30
8.5	函数式宏的典型用例	30
8.6	函数式宏的挑战	30
8.7	小结	31
9	错误处理与调试	32
9.1	错误处理的两条原则	32
9.2	<code>syn::Error</code> — 正确的错误处理	32
9.3	在 <code>derive</code> 宏中使用 <code>Error</code>	33
9.4	<code>Span</code> : 错误定位的生命线	34
9.5	属性解析中的错误	34
9.6	调试技巧	35
9.7	避免 <code>panic</code>	36
9.8	小结	37
10	测试过程宏	38
10.1	策略一: 集成测试	38
10.2	策略二: 用 <code>proc_macro2</code> 做单元测试	38
10.3	策略三: 编译失败测试 (<code>trybuild</code>)	39
10.4	策略四: 快照测试	40
10.5	测试策略总结	40
10.6	最佳实践	40
10.7	小结	41
11	实战: 从零实现 <code>derive(Builder)</code>	42
11.1	目标	42
11.2	项目结构	42
11.3	核心设计	42
11.4	解析阶段	43
11.5	代码生成阶段	44
11.6	完整的展开示例	45
11.7	进阶改进	46
11.8	常见陷阱	47
11.9	小结	47
12	进阶主题	48
12.1	卫生性 (<code>Hygiene</code>)	48
12.2	<code>Span</code> 的深层语义	48
12.3	模块化	49
12.4	编译时资源嵌入	49
12.5	发布过程宏 <code>crate</code>	50
12.6	过程宏的限制	50

12.7 未来方向	51
12.8 本质	51
12.9 小结	51
13 回顾与展望	52
13.1 回顾旅程	52
13.2 概念速查表	52
13.3 推荐阅读	52
13.4 最后的话	52

1 序章：重复代码的问题

想象一下这个场景。

某团队正在用 Rust 开发一个游戏引擎。他们需要为多种类型实现类似的功能——

- 为每个组件类型实现一个 `update` 方法
- 为每种错误类型实现 `Display` 和 `Error trait`
- 为每个配置结构体写 `Default` 和 `builder` 模式

他们写了很多重复的代码。

1.1 方案一：复制粘贴

最直接的办法——手动为每个类型写一遍。

```
#[derive(Debug)]
pub struct Player {
    pub name: String,
    pub health: i32,
}

impl Player {
    pub fn new(name: String, health: i32) -> Self {
        Self { name, health }
    }
}

impl fmt::Display for Player {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "Player({}, {})", self.name, self.health)
    }
}
```

再到 `Monster`、`Item`、`NPC`.....每个都手动写一遍。团队有 50 个组件类型。

方案	代价
复制粘贴 50 次	约 3000 行模板代码，改一个字段名要改 5 个地方

对于一次性原型，复制粘贴最快。但维护成本随着重复次数线性增长。

1.2 方案二：macro_rules!

Rust 自带的声明式宏可以处理简单的模板化场景：

```
macro_rules! impl_display {
    ($ty:ty, $($field:ident),+) => {
        impl fmt::Display for $ty {
            fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
                write!(f, stringify!($ty) "("
                    $( write!(f, "{}: {}", stringify!($field), self.$field) ),+
                    $( write!(f, ", ") ),+
                    write!(f, ")")
                )
            }
        }
    };
}
```

但 `macro_rules!` 有本质上的限制：

- 不能访问类型信息（不知道字段是 `String` 还是 `i32`）
- 不能根据类型做分支（不同类型用不同的格式化方式）
- 不能读取外部属性（`#[serde(rename = "...")]`）
- 语法表达能力有限，复杂逻辑很难写

`macro_rules!` 能处理简单的模板替换，但涉及类型信息的代码生成——它无能为力。

1.3 方案三：过程宏

Rust 的 过程宏（`procedural macros`）从根本上解决了这个问题。

它不是模板替换——它是把代码当作数据来操作：

- 接收一段 Rust 代码（以 `TokenStream` 的形式）
- 可以用任意 Rust 逻辑解析和处理它
- 输出一段新的 Rust 代码

```
// 过程宏是 TokenStream → TokenStream 的函数
#[proc_macro_derive(MyTrait)]
pub fn derive_my_trait(input: TokenStream) -> TokenStream {
    // 想做什么都行——遍历字段、读属性、访问文件、查数据库.....
}
```

过程宏打破了 `macro_rules!` 的所有限制：

能力	<code>macro_rules!</code>	过程宏	—— :-: :-:	访问类型信息	✗	✓	读取属性	✗	✓
条件分支	有限	✓	任意逻辑	自定义语法	✗	✓	文件 I/O	✗	✓

这就是 `serde` 的 `#[derive(Serialize)]`、`tokio` 的 `#[tokio::main]`、`sqlx` 的 `query!` 背后使用的技术。

1.4 本教程的目标

这本笔记不是 API 手册——它是一份探索记录。

我们从最基础的概念（“`TokenStream` 到底是什么？”）开始，一步步深入，直到能写出一个完整的 `#[derive(Builder)]` 宏。

每章围绕一个核心问题展开：

> 第一章：过程宏和 `macro_rules!` 有什么区别？ > 第二章：`TokenStream` 到底是什么？ > 第三章：怎么把 `TokenStream` 解析成有意义的结构？ > 第四章：怎么安全地生成代码？ >

准备好了吗？从第一章开始。

2 从 macro_rules! 到过程宏

✍ 实验记录： 第一站

2.1 两种宏的对比

Rust 有两种宏系统：声明式宏 (macro_rules!) 和 过程宏 (procedural macros)。

它们的本质区别：

特性	macro_rules!		过程宏		— :- :-		本质		模式匹配 + 替换		TokenStream
→	TokenStream 的函数		处理能力		匹配替换		任意 Rust 逻辑		类型信息		不可访问
	可解析类型签名		属性访问		不可访问		可读取 #[...] 属性		编译开销		极轻量
	有额外开销		宿主 crate		不需要额外依赖		必须是 proc-macro crate				

过程宏是 **编译期运行的函数**。它输入一段代码的 TokenStream，输出另一段 TokenStream。编译器在类型检查之前执行宏展开。

2.2 项目结构

创建一个过程宏 crate：

```
cargo new hello_proc_macro --lib
cd hello_proc_macro
```

编辑 Cargo.toml：

```
[lib]
proc-macro = true

[dependencies]
quote = "1.0"
syn = { version = "2.0", features = ["full"] }
```

定义

过程宏 crate 必须在 Cargo.toml 中设置 proc-macro = true。

这个标志告诉编译器：这个 crate 不是普通库，而是在编译期运行的代码生成器。

它只能导出过程宏函数，不能导出普通函数供运行时使用。

2.3 第一个过程宏

最简单的过程宏——什么都不做，直接返回输入：

```
// src/lib.rs
use proc_macro::TokenStream;

#[proc_macro]
pub fn identity(input: TokenStream) -> TokenStream {
    input
}
```

创建一个测试工程来验证：

```
// tests/test.rs - 或者另一个 crate
use hello_proc_macro::identity;
```

```
identity! {
    fn hello() {
        println!("Hello from proc macro!");
    }
}

fn main() {
    hello(); // 输出: Hello from proc macro!
}
```

概念

过程宏的工作流程：

1. 编译器解析源代码，产生 `TokenStream`
2. 把 `TokenStream` 传给过程宏函数
3. 过程宏运行（编译期），返回新的 `TokenStream`
4. 编译器用返回的 `TokenStream` 替换宏调用
5. 继续编译（类型检查、代码生成）

2.4 三种过程宏

Rust 有三种过程宏，用不同的属性标注：

定义

函数式宏 (Function-like macros)

`#[proc_macro]` - 像函数调用一样使用：`my_macro!(...)`

派生宏 (Derive macros)

`#[proc_macro_derive]` - 用于 `#[derive(MyTrait)]`

属性宏 (Attribute macros)

`#[proc_macro_attribute]` - 用于给函数/结构体加注解：`#[my_attr]`

2.5 函数式宏示例

```
use proc_macro::TokenStream;

#[proc_macro]
pub fn make_answer(_input: TokenStream) -> TokenStream {
    "fn answer() -> u32 { 42 }".parse().unwrap()
}
```

使用：

```
use my_macros::make_answer;

make_answer!();

fn main() {
    println!("{}", answer()); // 42
}
```

字符串 `.parse::<TokenStream>()` 是最简单的代码生产方式——但不推荐用于复杂场景（后面会讲 `quote!`）。

2.6 macro_rules! 做不到的事

```
// 用过程宏做类型感知的代码生成
#[proc_macro_derive(MySerialize)]
pub fn derive_my_serialize(input: TokenStream) -> TokenStream {
    // 用 syn 解析出 DeriveInput
    // 遍历字段、读取类型、生成序列化代码
    // 如果字段是 Option<T>, 生成不同的处理方式
    // 如果字段有 #[serde(skip)] 属性, 跳过它
    // ...
}
```

这就是 `serde` 的 `#[derive(Serialize)]` 的工作原理——`macro_rules!` 完全做不到。

2.7 过程宏的代价

⚠ 注意

使用过程宏之前要知道的代价：

- **编译时间增加**：过程宏在编译期运行，而且是独立编译的
- **调试困难**：错误信息可能不直观，需要特殊工具（`cargo expand`）
- **只能返回代码**：不能和运行时交互
- **需要额外依赖**：`syn`、`quote` 几乎是必备的
- **卫生性处理**：需要注意标识符冲突问题（后面会讲）

2.8 小结

- `macro_rules!` 基于模式匹配——适合简单的模板化代码
- 过程宏是 `TokenStream → TokenStream` 的函数——可以做任意复杂逻辑
- 三种过程宏：函数式宏、派生宏、属性宏
- 过程宏 `crate` 必须设置 `proc-macro = true`
- 核心库：`proc_macro`（内置）、`syn`（解析）、`quote`（生成）
- 下一步：理解 `TokenStream` 到底是什么

3 TokenStream — 编译器眼中的代码

🔧 实验记录：第二站

TokenStream 是理解过程宏的**核心概念**——它是编译器表示源代码的内部数据结构。

它不是字符串。它是编译器经过词法分析后的输出——一串带有位置信息的 `token`。

3.1 TokenStream 的组成

TokenStream 包含一系列 **TokenTree**，每个 TokenTree 是以下四种之一：

TokenTree 类型	对应什么	示例	:- :- :-	Ident	标识符	foo、struct、_x
Punct	标点符号	+, ::, ->	Literal	字面量	42、"hello"、3.14	Group
括号分组	{ ... }, (...), [...]					

示例

对于这段代码：

```
fn add(x: i32) -> i32 { x + 1 }
```

编译器看到的 TokenStream 大致是：

```
Ident("fn"), Ident("add"), Group(() [
  Ident("x"), Punct(":"), Ident("i32")
]), Punct("->"), Ident("i32"), Group({ [
  Ident("x"), Punct("+"), Literal("1")
]})
```

() 和 {} 是 Group——它们包含了内部的 TokenStream。

3.2 打印 TokenStream

写一个宏来观察输入的 TokenStream 结构：

```
#[proc_macro]
pub fn print_tokens(input: TokenStream) -> TokenStream {
    for tt in input.clone() {
        match tt {
            TokenTree::Ident(ident) => eprintln!("Ident: {}", ident),
            TokenTree::Punct(punct) => {
                eprintln!("Punct: {}, (spacing={:?})", punct, punct.spacing())
            }
            TokenTree::Group(group) => {
                eprintln!("Group({:?}): [ {:#?} ]", group.delimiter(),
group.stream())
            }
            TokenTree::Literal(literal) => eprintln!("Literal: {}", literal),
        }
    }
    input
}
```

```
print_tokens! {
    fn hello(x: u32) -> u32 { x * 2 }
}
```

stderr 输出:

```
Group(Brace): [  
  Ident: "fn"  
  Ident: "hello"  
  Group(Paren): [  
    Ident: "x"  
    Punct: ":"  
    Ident: "u32"  
  ]  
  Punct: "->"  
  Ident: "u32"  
  ...  
]
```

🔍 核心直觉

TokenStream 是编译器的“母语”。

你的 Rust 代码经过词法分析 → TokenStream → AST → HIR → MIR → 机器码。

过程宏在 TokenStream 阶段介入——把输入 TokenStream 变换成输出 TokenStream。

3.3 Group: 括号里的世界

定义

Group 代表一对配对的括号 ((), [], { }) 及其内部的所有 token。

每个 Group 有:

- `delimiter()`: 括号类型 (Parenthesis、Bracket、Brace、None)
- `stream()`: 内部的 TokenStream

Group 在过程宏中非常重要——当你解析 Rust 代码时, 所有“内部”的东西都被包在 Group 里。fn foo(args) { body } 中, 参数列表在 (...) Group 里, 函数体在 {...} Group 里。

3.4 Ident、Punct、Literal

类型	关键方法	用途
Ident	<code>.to_string()</code>	读取标识符名称
Punct	<code>.as_char()</code> 、 <code>.spacing()</code>	读取符号, 判断是否有空格间隔
Literal	<code>.to_string()</code>	读取字面量文本表示

Punct 的 `spacing()` 返回 `Spacing::Alone` 或 `Spacing::Joint`:

- `->`: `-` 的 `spacing` = Joint (和 `>` 连在一起)
- `+ 1`: `+` 的 `spacing` = Alone (后面有空格)

3.5 TokenStream 是可解析的

TokenStream 是一个扁平化的 token 序列, Group 展开后形成树形结构。

在过程宏中, 你的工作就是:

1. 接收一段输入 TokenStream
2. 解析 它——转换成有意义的数据结构 (用 `syn`)
3. 处理 它——根据数据生成新的代码
4. 输出 新的 TokenStream (用 `quote`)

整个过程在编译器内部完成, 不产生运行时开销。

3.6 字符串与 TokenStream 的互相转换

```
// 字符串 → TokenStream
let ts: TokenStream = "fn answer() -> u32 { 42 }".parse().unwrap();

// TokenStream → 字符串（用于调试）
let s = ts.to_string();
```

不过日常开发中你不会这样写——你会用 `syn` 解析，用 `quote!` 生成。

⚠ 注意

直接用字符串拼接生成代码非常不推荐：

- 没有语法检查
- 容易产生无效的 Rust 代码
- 没有卫生性保护

后面会学 `quote!`——安全的代码生成方式。

3.7 小结

- `TokenStream` = 词法分析后的 `token` 序列
- 四种 `TokenTree`: `Ident` (标识符)、`Punct` (标点)、`Literal` (字面量)、`Group` (括号分组)
- `Group` 包含括号和内部 `TokenStream`——代码的嵌套结构由此体现
- 过程宏的流程：输入 `TokenStream` → 解析 → 处理 → 生成 → 输出 `TokenStream`
- 不要直接拼接字符串来生成代码——用 `syn` 和 `quote`

4 解析：用 `syn` 把 `TokenStream` 变成数据结构

 实验记录：第三站

`TokenStream` 是一串 `token`。但我们需要像 Rust 编译器一样“看懂”代码——知道这是一个结构体定义，它有 3 个字段，第一个字段类型是 `String`，等等。

`syn` 就是做这个的。



概念

`syn` 是一个 Rust 库，它能把 `TokenStream` 解析 (parse) 成 Rust 语法对应的数据结构。

就像 `serde_json` 把 JSON 字符串解析成 Rust 结构体，`syn` 把 Rust 源代码 `TokenStream` 解析成语法树 (AST)。

4.1 第一条解析代码

```
use proc_macro::TokenStream;
use syn::{parse_macro_input, DeriveInput};

#[proc_macro_derive(Hello)]
pub fn derive_hello(input: TokenStream) -> TokenStream {
    // parse_macro_input! 是 syn 提供的便捷宏
    // 它把 TokenStream 解析成 DeriveInput 结构体
    // 如果解析失败，会自动生成友好的编译错误
    let input = parse_macro_input!(input as DeriveInput);

    // 打印结构体名称到 stderr (编译时可见)
    eprintln!("Struct name: {}", input.ident);

    // 暂时返回空 TokenStream
    TokenStream::new()
}
```

4.2 `DeriveInput`

定义

`DeriveInput` 是 `derive` 宏中最常用的解析类型。它代表用 `#[derive(...)]` 标记的数据类型定义。

包含：

- `ident: Ident` — 结构体/枚举的名称
- `attrs: Vec<Attribute>` — 外部属性 (如 `#[serde(...)]`)
- `data: Data` — 具体的数据定义 (结构体/枚举/联合体)
- `generics: Generics` — 泛型参数

示例

给定这个输入：

```
#[derive(Hello)]
#[serde(rename_all = "camelCase")]
pub struct Player {
    pub name: String,
    #[hello(greeting = "Hi")]
```

```
pub level: u32,
}
```

syn 解析出的 `DeriveInput` 结构:

```
DeriveInput {
  ident: Ident("Player"),
  attrs: [Attribute { path: "serde", ... }],
  data: Data::Struct(DataStruct {
    struct_token: Struct,
    fields: Fields::Named(FieldsNamed {
      named: [
        Field { ident: Some("name"), ty: Type::Path("String"), attrs:
[] },
        Field { ident: Some("level"), ty: Type::Path("u32"),
          attrs: [Attribute { path: "hello", ... }] },
      ],
    }),
  }),
  generics: Generics { params: [], ... },
}
```

syn 把扁平的 token 整理成了有意义的层级结构!

4.3 解析不同的数据类型

`Data` 枚举代表了 Rust 的数据类型定义:

```
pub enum Data {
  Struct(DataStruct), // struct { ... }
  Enum(DataEnum),     // enum { ... }
  Union(DataUnion),   // union { ... }
}
```

每个变体都包含对应的字段信息:

```
pub enum Fields {
  Named(FieldsNamed), // struct { x: i32, y: String }
  Unnamed(FieldsUnnamed), // struct(i32, String)
  Unit, // struct Nothing;
}
```

`derive` 宏可以通过匹配这些枚举来判断输入的结构类型, 分别处理。

4.4 不只是 `DeriveInput`

syn 几乎能解析所有 Rust 语法结构:

| syn 类型 | 对应 Rust 语法 | `|:-|:-|` | `ItemFn` | `fn foo() { ... }` | | `ItemImpl` | `impl Foo for Bar { ... }` | | `ItemTrait` | `trait Foo { ... }` | | `Type` | 任何类型表达式 (`Vec<u32>`、`&str`) | | `Expr` | 任何表达式 (`a + b`、`foo()`) | | `Lit` | 字面量 (`42`、`"str"`、`true`) | | `Pat` | 模式 (`Some(x)`、`_`) | | `Attribute` | `#[foo(...)]` |

这意味着我们可以解析函数体、`trait` 定义、甚至任意表达式。

4.5 属性解析：读取 #[...]

概念

属性（attributes）是过程宏和外部世界通信的主要方式。

`#[serde(skip)]` 告诉 `serde` “跳过这个字段”。`#[builder(default)]` 告诉 `builder` 宏“这个字段用默认值”。

```
use syn::{Attribute, Meta};

fn parse_attributes(attrs: &[Attribute]) {
    for attr in attrs {
        match &attr.meta {
            Meta::Path(path) => {
                // 简单属性：#[skip]
            }
            Meta::List(meta_list) => {
                // 带参数的属性：#[serde(rename = "foo")]
                // meta_list.tokens 包含括号内的 token
            }
            Meta::NameValue(meta_name_value) => {
                // 键值对：#[doc = "documentation"]
            }
        }
    }
}
```

4.6 Parse trait: 自定义解析

对于自定义 DSL（比如 `sql!` 宏），可以为自定义类型实现 `Parse trait`：

```
use syn::parse::{Parse, ParseStream};

struct SqlQuery {
    table: syn::Ident,
    columns: Vec<syn::Ident>,
}

impl Parse for SqlQuery {
    fn parse(input: ParseStream) -> syn::Result<Self> {
        // 解析 "SELECT col1, col2 FROM table"
        input.parse::<syn::Token![SELECT]>()?;
        let mut columns = Vec::new();
        while !input.peek(syn::Token![FROM]) {
            columns.push(input.parse()?);
            if input.peek(syn::Token![,]) {
                input.parse::<syn::Token![,]>()?;
            }
        }
        input.parse::<syn::Token![FROM]>()?;
        let table = input.parse()?;
        Ok(SqlQuery { table, columns })
    }
}
```

过程宏不局限于标准 Rust 语法——可以用 Parse 定义自己的语法规则。

4.7 syn 的 features

```
[dependencies]
syn = { version = "2.0", features = [
    "full",          # 完整语法解析 (ItemFn、Expr 等)
    "extra-traits",  # 为语法树实现 Debug、Clone、Eq 等
    "visit",         # 语法树遍历
    "fold",          # 语法树变换
] }
```

- 只写 derive 宏? features = ["derive"] 就够了
- 写属性宏或函数式宏? 需要 features = ["full"]
- features = ["default"] 包含 derive、parsing、printing 等基础功能

4.8 动手验证

```
use syn::{DeriveInput, Data, Fields};

fn describe_struct(input: &DeriveInput) {
    match &input.data {
        Data::Struct(data) => {
            let fields = &data.fields;
            println!("Struct `{}` has {} field(s):", input.ident, fields.len());
            for (i, field) in fields.iter().enumerate() {
                let name = field.ident.as_ref()
                    .map(|id| id.to_string())
                    .unwrap_or_else(|| format!("field_{}", i));
                let ty = quote::quote! { #field.ty }.to_string();
                println!(" - {}: {}", name, ty);
            }
        }
        Data::Enum(data) => {
            println!("Enum `{}` has {} variant(s):", input.ident,
                data.variants.len());
            for variant in &data.variants {
                println!(" - {}", variant.ident);
            }
        }
        _ => {}
    }
}
```

4.9 小结

- syn 把 TokenStream 解析成 Rust 语法树
- DeriveInput 是 derive 宏的入口数据结构
- Data 枚举区分为结构体/枚举/联合体
- Fields 枚举区分为命名/未命名/单元字段
- Attribute 让宏能读取 #[...] 参数
- 通过实现 Parse trait 可以解析自定义语法
- syn 的 feature flags 控制编译内容

5 生成代码：quote! 魔法

 实验记录：第四站

解析搞定了——`syn` 把 `TokenStream` 变成我们可以操作的数据结构。现在反过来：从数据结构如何生成新的 `TokenStream`？

最直接的方式是字符串拼接，但不推荐：

```
format!("impl Hello for {} {{ ... }}", name)
```

问题：

1. 没有语法检查——拼错了编译器才知道
2. 标识符冲突风险——如果用户有个类型叫 `impl` 呢？
3. 没有卫生性——生成的临时变量可能和用户代码冲突
4. 代码丑陋、难以维护

`quote` 库提供了更好的方式。



概念

`quote!` 宏让你在 Rust 代码中书写“模板”代码，模板中的 `#var` 会被替换为实际值。它生成的 `TokenStream` 是语法安全的、卫生的，而且写起来像在写普通的 Rust 代码。

5.1 第一个 quote! 示例

```
use quote::quote;

let name = syn::Ident::new("Player", proc_macro2::Span::call_site());
let tokens = quote! {
    impl #name {
        pub fn new() -> Self {
            Self {}
        }
    }
};
// tokens 类型是 proc_macro2::TokenStream
// 可以转为 proc_macro::TokenStream: tokens.into()
```

展开后相当于：

```
impl Player {
    pub fn new() -> Self {
        Self {}
    }
}
```



核心直觉

`quote!` 的魔法：你在 `quote! { ... }` 里写的是 Rust 代码，但它不会编译——它被编译成了生成这段 Rust 代码的代码。

`#var` 插值把 Rust 值嵌入到生成的代码中。

5.2 插值方式

| 语法 | 类型 | 效果 | |:-|:-|:-| | #var | Ident / TokenStream / ToTokens | 插入一个值 | | #(#var)* | 迭代器 | 重复插值 | | #(#var),* | 迭代器 | 重复插值, 逗号分隔 | | #(#var),* , | 迭代器 | 重复插值, 逗号分隔 + 尾部逗号 |

5.3 为结构体生成 getter 方法

示例

```
use syn::{Data, Fields, DeriveInput};
use quote::quote;

fn generate_getters(input: &DeriveInput) -> proc_macro2::TokenStream {
    let name = &input.ident;

    let fields = match &input.data {
        Data::Struct(data) => &data.fields,
        _ => panic!("Expected a struct"),
    };

    let getters = fields.iter().map(|field| {
        let field_name = &field.ident;
        let field_type = &field.ty;

        quote! {
            pub fn #field_name(&self) -> &#field_type {
                &self.#field_name
            }
        }
    });

    quote! {
        impl #name {
            #(#getters)*
        }
    }
}
```

输入 `struct Point { x: i32, y: i32 }`, 输出:

```
impl Point {
    pub fn x(&self) -> &i32 { &self.x }
    pub fn y(&self) -> &i32 { &self.y }
}
```

5.4 条件生成

quote! 支持条件插入。常用 `Option<TokenStream>` 模式:

```
let extra_impl = if has_debug {
    Some(quote! { impl std::fmt::Debug for #name { ... } })
} else {
    None // None 表示不插入任何代码
};
```

5.5 proc_macro2 与 proc_macro

quote! 返回的是 `proc_macro2::TokenStream`, 不是 `proc_macro::TokenStream`。

定义

`proc_macro2` 是 `proc_macro` 的跨平台封装。

`proc_macro`	`proc_macro2`	`	-	-	`	只能在过程宏 `crate` 中使用	普通 `crate` 也能用
	不是 `Send + Sync`	是 `Send + Sync`		只能操作来自编译器的 `TokenStream`			
可以创建任意 `TokenStream`		不能用于测试	可在单元测试中使用				

典型写法—入口用 `proc_macro`, 内部全部用 `proc_macro2`:

```
#[proc_macro_derive(Hello)]
pub fn derive_hello(input: proc_macro::TokenStream) -> proc_macro::TokenStream {
    let input: proc_macro2::TokenStream = input.into();           // 转
    let derive_input = syn::parse2::<syn::DeriveInput>(input).unwrap(); // 解析
    let output = quote! { /* ... */ };                             // 生成
    output.into()                                                  // 转回
}
```

5.6 ToTokens trait

任何实现了 `ToTokens` trait 的类型都可以用 `#var` 插入到 `quote!` 中。

- `Ident`、`Literal`、`Type`、`Expr` 等 `syn` 类型都实现了
- `String`、`i32` 等基础类型也实现了（生成字面量）
- 可以为自定义类型实现 `ToTokens`

```
use quote::{ToTokens, TokenStreamExt};

struct Wrapper(syn::Type);

impl ToTokens for Wrapper {
    fn to_tokens(&self, tokens: &mut proc_macro2::TokenStream) {
        // 在类型外面包一层 Wrapper<>
        // self.0.to_tokens(tokens);
        tokens.append(Ident::new("Wrapper", span));
    }
}
```

5.7 常见陷阱

⚠ 注意

引用与借用:

- `#(fields),*` 中的 `fields` 如果是迭代器, `quote!` 会消费它
- 如果需要多次使用, 用 `.collect::<Vec<_>>()` 先收集

Span 控制:

- `quote!` 默认使用当前调用位置的 `span`
- 可以用 `quote_spanned! { span => ... }` 指定 `span` 以改善错误定位

5.8 小结

- `quote!` 宏让在代码中写“模板”, `#var` 做插值

- `#![iter]*` 用于循环插入代码
- `proc_macro2` 是跨平台封装, `syn` 和 `quote` 都基于它
- `ToTokens trait` 定义值如何转换为 `TokenStream`
- `quote_spanned!` 控制生成代码的 `span` (错误定位)
- 永远用 `quote!` 而不是字符串拼接来生成代码

6 派生宏 – 自动实现 Trait

 实验记录： 第五站

Rust 中最常见的过程宏就是 `#[derive(...)]`。

`Serde` 用它生成序列化代码，`clap` 用它从结构体生成命令行参数解析器，`thiserror` 用它生成 `Error` 实现。

6.1 从一个简单的 `trait` 开始

首先定义目标 `trait`：

```
// hello_macro_def/src/lib.rs
pub trait HelloMacro {
    fn hello_macro();
}
```

然后写过程宏，让用户能这样用：

```
use hello_macro::HelloMacro;
use hello_macro_derive::HelloMacro;

#[derive(HelloMacro)]
struct Pancakes;

fn main() {
    Pancakes::hello_macro(); // 输出: "Hello from Pancakes!"
}
```

6.2 派生宏的签名

```
// hello_macro_derive/src/lib.rs
use proc_macro::TokenStream;
use quote::quote;
use syn::{parse_macro_input, DeriveInput};

#[proc_macro_derive(HelloMacro)]
pub fn hello_macro_derive(input: TokenStream) -> TokenStream {
    let input = parse_macro_input!(input as DeriveInput);
    let name = &input.ident;

    let expanded = quote! {
        impl hello_macro::HelloMacro for #name {
            fn hello_macro() {
                println!("Hello from {}!", stringify!(#name));
            }
        }
    };

    expanded.into()
}
```

核心直觉

`Derive` 宏的核心流程：

1. 编译器遇到 `#[derive(MyTrait)]` 时, 把整个类型定义作为 `TokenStream` 传给宏
2. 宏用 `syn` 解析成 `DeriveInput`
3. 宏用 `quote!` 生成 `impl Trait for Type { ... }` 块
4. 编译器把生成的代码拼接到类型定义的后面

6.3 处理泛型

如果结构体有泛型参数, 不能简单粗暴地生成 `impl` 块——必须正确包含泛型约束。

```
#[derive(HelloMacro)]
struct Container<T: Display> {
    value: T,
}
```

定义

`DeriveInput` 包含 `generics: Generics` 字段。

`Generics` 提供了 `split_for_impl()` 方法, 返回三个部分:

- `impl_generics: <T: Display>` — 用于 `impl` 关键字后面
- `ty_generics: <T>` — 用于类型名称后面
- `where_clause`: 如果有 `where` 约束, 包含它

```
let (impl_generics, ty_generics, where_clause) =
input.generics.split_for_impl();

let expanded = quote! {
    impl #impl_generics hello_macro::HelloMacro for #name #ty_generics
#where_clause {
        fn hello_macro() {
            println!("Hello from {}!", stringify!(#name));
        }
    }
};
```

输出:

```
impl<T: Display> HelloMacro for Container<T> {
    fn hello_macro() { /* ... */ }
}
```

6.4 访问字段

```
use syn::{Data, Fields};

let fields = match &input.data {
    Data::Struct(data) => &data.fields,
    _ => panic!("HelloMacro only supports structs"),
};

let field_names: Vec<_> = fields.iter().map(|f| &f.ident).collect();
let field_types: Vec<_> = fields.iter().map(|f| &f.ty).collect();
```

```
let expanded = quote! {
    impl #impl_generics #name #ty_generics #where_clause {
        pub fn field_names() -> Vec<&'static str> {
            vec![(stringify!(#field_names)),*]
        }
        pub fn field_types() -> Vec<&'static str> {
            vec![(stringify!(#field_types)),*]
        }
    }
};
```

概念

字段的 `ident` 是 `Option<Ident>`:

- 命名结构体: `struct Foo { x: i32 } → Some("x")`
- 元组结构体: `struct Foo(i32, String) → None`
- 单元结构体: `struct Foo; → 没有字段`

元组结构体用索引访问: `self.0`、`self.1`.....

6.5 带属性的派生宏

让用户通过属性自定义行为:

```
#[derive>HelloMacro)]
#[hello_macro(greeting = "Hola")]
struct Person {
    name: String,
    #[hello_macro(skip)]
    age: u32,
}
```

关键在于 `#[proc_macro_derive]` 的第二个参数:

```
#[proc_macro_derive>HelloMacro, attributes(hello_macro))]
pub fn hello_macro_derive(input: TokenStream) -> TokenStream {
    // attributes(hello_macro) 告诉编译器:
    // 这个 derive 宏会消费 #[hello_macro(...)] 属性
    // 这样编译器不会报"不认识这个属性"
}
```

解析自定义属性:

```
fn parse_greeting(attrs: &[syn::Attribute]) -> Option<String> {
    for attr in attrs {
        if attr.path().is_ident("hello_macro") {
            let mut greeting = None;
            attr.parse_nested_meta(|meta| {
                if meta.path.is_ident("greeting") {
                    greeting =
                        Some(meta.value()?.parse:::<syn::LitStr>()?.value());
                }
            });
        }
    }
}
```

```

        Ok(())
    }).ok()?;
    return greeting;
}
}
None
}

```

6.6 完整的模式

```

let greeting = parse_greeting(&input.attrs)
                .unwrap_or_else(|| "Hello".to_string());

let expanded = quote! {
    impl #impl_generics #name #ty_generics #where_clause {
        fn hello_macro() {
            println!("{}", from {}!", #greeting, stringify!(#name));
        }
    }
};

```

6.7 Derive 宏的设计模式

所有 derive 宏都遵循同样的模式：

1. 用 `parse_macro_input!` 解析 `DeriveInput`
2. 用 `split_for_impl()` 处理泛型
3. 遍历 `data` 中的字段/变体
4. 读取 `attrs` 中的自定义属性
5. 用 `quote!` 生成 `impl Trait for Type` 代码
6. 用 `Result` 模式处理错误 (`syn::Error` $\rightarrow$ `to_compile_error()`)

这就是 `serde`、`clap`、`thiserror` 等所有 `derive` 宏的工作方式。

6.8 小结

- `Derive` 宏签名: `#[proc_macro_derive(Name)]`, 接收 `TokenStream`, 返回 `TokenStream`
- 用 `syn::DeriveInput` 解析类型定义
- 用 `generics.split_for_impl()` 正确处理泛型
- 用 `input.data` 获取字段信息
- 用 `attributes(...)` 注册宏消费的自定义属性
- `parse_nested_meta` 是解析属性的利器
- 核心模式: 解析 $\rightarrow$ 遍历 $\rightarrow$ 读取属性 $\rightarrow$ `quote!` 生成

7 属性宏 — 函数的注解

 实验记录： 第六站

Derive 宏只能用于类型定义。但如果我们想注解一个 **函数** 呢？比如在所有函数调用前后插入日志？这就是 **属性宏** (attribute macros) 的用途。

7.1 属性宏的结构

定义

属性宏用 `#[proc_macro_attribute]` 标注，接收 **两个** 参数：

```
#[proc_macro_attribute]
pub fn my_attribute(
    attr: TokenStream,    // 属性本身的参数
    item: TokenStream,    // 被注解的项
) -> TokenStream {
    // ...
}
```

- attr: `#[my_attribute(args...)]` 中的 `args...`
- item: 被注解的整个函数/结构体/模块定义

7.2 示例: `log_call`

在每次函数调用前后插入日志：

```
// 用户代码
#[log_call]
fn add(a: i32, b: i32) -> i32 {
    a + b
}

fn main() {
    add(3, 4); // 输出: "Calling add(3, 4)"
}
```

实现：

```
use proc_macro::TokenStream;
use quote::quote;
use syn::{parse_macro_input, ItemFn};

#[proc_macro_attribute]
pub fn log_call(attr: TokenStream, item: TokenStream) -> TokenStream {
    let input_fn = parse_macro_input!(item as ItemFn);
    let fn_name = &input_fn.sig.ident;
    let fn_vis = &input_fn.vis;
    let fn_sig = &input_fn.sig;
    let fn_block = &input_fn.block;

    let arg_names: Vec<_> = input_fn.sig.inputs.iter().map(|arg| {
        match arg {
            syn::FnArg::Typed(pat_type) => &pat_type.pat,
            _ => panic!("Unexpected argument type"),
        }
    });

    let log_call_code = quote! {
        println!("Calling {}({})", fn_name, arg_names.join(", "));
        fn_block
    };

    quote! {
        #attr
        #fn_vis fn #fn_name #fn_sig {
            #log_call_code
        }
    }
}
```

```

    }
  }).collect();

  let expanded = quote! {
    #fn_vis #fn_sig {
      eprintln!("Calling {} with ({})",
        stringify!(#fn_name),
        stringify!(#(#arg_names),*))
    };
    let result = #fn_block;
    eprintln!("{}", stringify!(#fn_name), &result);
    result
  };

  expanded.into()
}

```

🔍 核心直觉

属性宏的工作方式：

1. 编译器看到 `#[log_call] fn add(...) { ... }`
2. 把整个函数定义传给宏
3. 宏输出替换原来的函数定义
4. 宏可以包一层、改签名、甚至完全换成不同的代码

7.3 用 `attr` 参数做配置

```

#[log_call(level = "warn")]
fn dangerous_op() {
    // ...
}

```

```

#[proc_macro_attribute]
pub fn log_call(attr: TokenStream, item: TokenStream) -> TokenStream {
    // attr 包含了 level = "warn" 这部分 token
    // 用 syn 解析 attr 参数
    if attr.is_empty() {
        // 默认行为
    } else {
        // 解析配置
    }
    // ...
}

```

7.4 典型用例

| 宏 | 用途 | |:-|:-| | `#[tokio::main]` | 将异步 `main` 函数包装进 `runtime` | | `#[test]` | 标记测试函数 | | `#[instrument] (tracing)` | 自动注入跟踪 `span` | | `#[must_use]` | 风格的 `lint` | 编译期检查 |

7.5 属性宏 vs Derive 宏

特性	Attribute Macro	Derive Macro	:- :-: :-:	输入	属性参数 + 被注解项
整个类型定义			输出	替换被注解项（完全改变）	追加 impl 块
函数、结构体、枚举、模块		结构体、枚举、联合体		灵活性	极高 中等
					典型用例
#[tokio::main]		#[derive(Serialize)]			

7.6 常见陷阱

⚠ 注意

属性宏展开时会完全替换被注解的项。

如果只是想加东西（而不是替换），必须在输出中包含原始代码：

```
// ❌ 错误：忘记了原始函数体
quote! {
    eprintln!("before");
    // 忘了 include 原始函数
    eprintln!("after");
}

// ✅ 正确：保留原始函数体
let original_body = &input_fn.block;
quote! {
    eprintln!("before");
    #original_body
    eprintln!("after");
}
```

7.7 小结

- 属性宏：#[proc_macro_attribute]，接收 (attr, item)，返回替换后的代码
- 可以应用于函数、结构体、枚举、模块等
- attr 参数让宏可配置
- 属性宏会完全替换被注解的项——不要忘了包含原始代码
- 典型用途：日志、拦截、包装、代码转换
- 和 Python 装饰器类似，但发生在编译时，没有运行时开销

8 函数式宏 — 自定义 DSL

✍ 实验记录：第七站

Derive 宏扩展类型，属性宏注解函数——但还有第三种：函数式宏，它可以创造全新的语法。

`sqlx::query!("SELECT * FROM users")` 就是一个典型——嵌入在 Rust 中的 SQL，在编译时被解析和验证。

8.1 函数式宏的签名

定义

函数式宏用 `#[proc_macro]` 标注，只接收一个参数：

```
#[proc_macro]
pub fn my_macro(input: TokenStream) -> TokenStream {
    // input 是 my_macro!(...) 中 ... 的 TokenStream
}
```

使用方式：

```
my_macro!(任何 Rust token 或自定义语法);
```

8.2 案例：简化的 json! 宏

实现一个简化版 `json!` 宏，把 JSON 字面量转换为 Rust 表达式：

```
let data = json!({
    "name": "Rust",
    "year": 2015,
    "tags": ["systems", "performance"]
});
```

因为输入是合法的 Rust token（字面量、方括号、花括号），所以可以用 `syn` 的通用解析器。

```
use syn::parse::{Parse, ParseStream};
use syn::{LitStr, LitInt, Token};

enum JsonValue {
    Str(LitStr),
    Num(LitInt),
    Array(Vec<JsonValue>),
    Object(Vec<(String, JsonValue)>),
}

impl Parse for JsonValue {
    fn parse(input: ParseStream) -> syn::Result<Self> {
        if input.peek(LitStr) {
            input.parse().map(JsonValue::Str)
        } else if input.peek(LitInt) {
            input.parse().map(JsonValue::Num)
        } else if input.peek(Token![{]) {
            let content;
            syn::braced!(content in input);
            let mut pairs = Vec::new();
        }
    }
}
```

```

        while !content.is_empty() {
            let key: LitStr = content.parse()?;
            content.parse::

```



概念

syn 提供的括号解析辅助宏:

- `syn::braced!(content in input)` – 匹配 `{...}`, 返回内容的 `ParseStream`
- `syn::bracketed!(content in input)` – 匹配 `[...]`
- `syn::parenthesized!(content in input)` – 匹配 `(...)`

8.3 生成代码

```

fn generate_json(value: &JsonValue) -> proc_macro2::TokenStream {
    match value {
        JsonValue::Str(lit) => {
            quote! { #lit.into() }
        }
        JsonValue::Num(lit) => {
            quote! { #lit.into() }
        }
        JsonValue::Array(items) => {
            let items_ts: Vec<_> = items.iter().map(generate_json).collect();
            quote! { vec! [ #(items_ts),* ] }
        }
        JsonValue::Object(pairs) => {
            let keys: Vec<_> = pairs.iter().map(|(k, _)| {
                syn::LitStr::new(k, proc_macro2::Span::call_site())
            }).collect();
            let values_ts: Vec<_> = pairs.iter()
                .map(|(_, v)| generate_json(v))
            .collect();
            quote! {
                {
                    let mut obj = Object::new();
                    #(obj.insert(#keys[#i], #values_ts[#i]);)
                    obj
                }
            }
        }
    }
}

```

```

        .collect();
    quote! {
        maplit::btreemap! {
            #( #keys.into() => #values_ts ),*
        }
    }
}
}
}

```

8.4 解析自定义 DSL

函数式宏不限于 Rust 语法——你可以定义自己的 DSL。

示例

一个简单的 HTML 模板宏:

```
let name = "World";
let greeting = html! {
  <div class="container">
    <h1>"Hello, " (name) "!"</h1>
  </div>
};
```

这里 `<` 和 `>` 在 `TokenStream` 中是 `Punct`, `div` 是 `Ident`。解析器需要手动识别 `< ident attr... >` 的模式。

```
enum HtmlNode {
    Element { tag: String, attrs: Vec<(String, String)>, children:
Vec<HtmlNode> },
    Text(String),
    Expr(proc_macro2::TokenStream),
}

fn parse_html(input: proc_macro2::TokenStream) -> Vec<HtmlNode> {
    let tokens: Vec<_> = input.into_iter().collect();
    // 遍历 token, 识别 < > 结构
    // 遇到 ( ... ) 时, 内部当作 Rust 表达式
    vec![]
}
```

8.5 函数式宏的典型用例

宏	用途	:-: -	lazy_static!	声明懒静态变量	sqlx::query!	编译时 SQL
解析和验证	html!	JSX 风格 HTML 模板	json! (serde)	JSON 字面量转 Rust		
值	regex!	编译期正则编译				

8.6 函数式宏的挑战

⚠️ **注意**

函数式宏虽然灵活，但也有代价：

1. **解析是手动的**——不同于 `derive` 宏有 `DeriveInput` 这种现成结构
2. **错误信息可能不友好**——需要手动处理 `span`

⚠️ **注意**

函数式宏虽然灵活，但也有代价：

1. **解析是手动的**——不同于 `derive` 宏有 `DeriveInput` 这种现成结构
2. **错误信息可能不友好**——需要手动处理 `span`

- ⚠️ **注意**

函数式宏虽然灵活，但也有代价：

 1. **解析是手动的**——不同于 `derive` 宏有 `DeriveInput` 这种现成结构
 2. **错误信息可能不友好**——需要手动处理 `span`

3. 语法不一定是合法 **Rust**——`<div>` 在普通 Rust 中是运算符，在 `html!` 中是标签
4. 维护成本高——自定义语法需要自己写解析器

函数式宏展示了过程宏的终极形态：在 Rust 中嵌入任意 DSL。但大多数场景下，`derive` 和 `attribute` 宏已经足够——它们约束在 Rust 语法内，更安全、更容易维护。

8.7 小结

- 函数式宏：`#[proc_macro]`，接收一个 `TokenStream`，返回一个 `TokenStream`
- 输入可以是任何 token 序列——不受 Rust 语法约束
- 可以用 `syn::parse::Parse trait` + 辅助宏 (`braced`、`bracketed`) 解析
- 适合嵌入式 DSL：html、sql、json、regex
- 灵活性最高，但解析工作和错误处理也更复杂
- 如果 `derive` 或 `attribute` 宏能满足需求，优先用它们

9 错误处理与调试

实验记录：第八站

过程宏出错时，编译器只会给出很有限的信息：

```
error: proc macro panicked
--> src/main.rs:10:1
   |
10 | #[derive(MyMacro)]
   | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
   |
   = help: message: called `Result::unwrap()` on an `Err` value: ...
```

没有行号、没有上下文——错误指向宏调用的位置，而不是宏内部的具体问题。

这意味着错误处理对过程宏来说比普通代码更重要。

9.1 错误处理的两条原则

概念

过程宏中的错误处理：

1. 永远不要 **unwrap**。使用 `syn::Result` 和 `?` 操作符传播错误
2. 尽量让错误指向用户的代码位置。使用 `Span` 提供精确的行列信息

9.2 `syn::Error` — 正确的错误处理

```
use syn::Error;

// ✗ 错误做法
fn parse_field(input: ParseStream) -> syn::Result<Field> {
    let name = input.parse::<Ident>().unwrap();
    // ...
}

// ✓ 正确做法
fn parse_field(input: ParseStream) -> syn::Result<Field> {
    let name: Ident = input.parse()?; // 传播错误
    // ...
}
```

定义

`syn::Error` 是过程宏的标准错误类型，有两个关键能力：

1. 关联一个 `Span`——指向用户代码中的具体位置
2. 可以组合多个错误一起报告

```
use syn::Error;
use proc_macro2::Span;

let error = Error::new(span, "expected a string literal");
```

```
let mut errors = Error::new(span1, "error 1");
errors.combine(Error::new(span2, "error 2"));
```

9.3 在 derive 宏中使用 Error

```
use proc_macro::TokenStream;
use syn::{parse_macro_input, DeriveInput, Error};

#[proc_macro_derive(MyTrait)]
pub fn derive_mytrait(input: TokenStream) -> TokenStream {
    let input = parse_macro_input!(input as DeriveInput);

    match derive_mytrait_impl(&input) {
        Ok(tokens) => tokens.into(),
        Err(err) => err.to_compile_error().into(),
    }
}

fn derive_mytrait_impl(input: &DeriveInput)
-> syn::Result<proc_macro2::TokenStream>
{
    let name = &input.ident;

    let data = match &input.data {
        syn::Data::Struct(data) => data,
        _ => {
            return Err(Error::new_spanned(
                input,
                "MyTrait can only be derived for structs",
            ));
        }
    };

    if data.fields.is_empty() {
        return Err(Error::new_spanned(
            name,
            "MyTrait requires at least one field",
        ));
    }

    Ok(quote! { /* ... */ })
}
```

核心直觉

`err.to_compile_error()` 是把 `syn::Error` 转化为 `compile_error!()` 宏调用的关键方法。它生成类似这样的代码：

```
compile_error!("MyTrait can only be derived for structs");
```

并带有正确的 `span` 信息，让编译器精准报错。

9.4 Span: 错误定位的生命线

概念

Span 代表源代码中的一段位置。

当你在 `syn` 中解析代码时，每个 `token` 都携带了它在源文件中的 `span` 信息。

- `Error::new(span, msg)` – 创建一个指向 `span` 位置的错误
- `Error::new_spanned(value, msg)` – 自动使用 `value` 的 `span`

```
fn check_field(field: &syn::Field) -> syn::Result<()> {
    let field_name = field.ident.as_ref()
        .ok_or_else(|| Error::new_spanned(
            field,
            "tuple struct fields are not supported",
        ))?;

    if !is_valid_type(&field.ty) {
        return Err(Error::new_spanned(
            &field.ty,
            format!("unsupported type `{}`", quote!(#field.ty)),
        ));
    }

    Ok(())
}
```

当错误发生时，编译器的错误信息会指向具体的字段或类型，而不是宏调用的位置。

9.5 属性解析中的错误

```
fn parse_my_attr(attrs: &[syn::Attribute]) -> syn::Result<MyAttr> {
    for attr in attrs {
        if attr.path().is_ident("my_attr") {
            return attr.parse_args::<MyAttr>();
            //      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ 自动使用 attr 的 span
        }
    }
    Ok(MyAttr::default())
}

struct MyAttr { name: Option<String> }

impl syn::parse::Parse for MyAttr {
    fn parse(input: syn::parse::ParseStream) -> syn::Result<Self> {
        let mut name = None;
        if !input.is_empty() {
            let ident: syn::Ident = input.parse()?;
            if ident != "name" {
                return Err(syn::Error::new(ident.span(),
                    "expected `name`"));
            }
            input.parse::<syn::Token![=]>()?;
            let lit: syn::LitStr = input.parse()?;
            name = Some(lit.value());
        }
    }
}
```

```

    }
    Ok(MyAttr { name })
  }
}

```

9.6 调试技巧

9.6.1 1. cargo expand – 查看宏展开结果

```

cargo install cargo-expand
cargo expand # 展开所有宏，查看生成的代码

```

示例

```

$ cargo expand
struct Point { x: i32, y: i32 }

#[automatically_derived]
impl MyTrait for Point {
    fn method(&self) -> String {
        format!("Point({}, {})", self.x, self.y)
    }
}

```

9.6.2 2. eprintln! – 打印调试信息

过程宏在编译期运行，打印到 `stderr` 会在编译输出中显示：

```

#[proc_macro_derive(MyTrait)]
pub fn my_derive(input: TokenStream) -> TokenStream {
    eprintln!("=== Debug ===");
    eprintln!("Input: {}", input);
    // ...
}

```

9.6.3 3. 使用 `syn::visit` 遍历语法树

```

use syn::visit::{self, Visit};

struct FieldVisitor {
    fields: Vec<String>,
}

impl<'ast> Visit<'ast> for FieldVisitor {
    fn visit_field(&mut self, field: &'ast syn::Field) {
        if let Some(ref ident) = field.ident {
            self.fields.push(ident.to_string());
        }
        visit::visit_field(self, field);
    }
}

```

```
let mut visitor = FieldVisitor { fields: vec![] };
visitor.visit_derive_input(&input);
println!("Found fields: {:?}", visitor.fields);
```

9.6.4 4. 单元测试

用 `proc_macro2` 而非 `proc_macro`:

```
#[cfg(test)]
mod tests {
    use super::*;
    use syn::parse_quote;

    #[test]
    fn test_derive() {
        let input: syn::DeriveInput = parse_quote! {
            struct Point { x: i32, y: i32 }
        };
        let result = derive_my_trait_impl(&input).unwrap();
        let output = result.to_string();
        assert!(output.contains("impl MyTrait for Point"));
    }
}
```

9.7 避免 panic

⚠ 注意

永远不要在过程宏中 `panic!`

```
// ❌ 不要这样做
#[proc_macro_derive(Bad)]
pub fn bad_derive(input: TokenStream) -> TokenStream {
    let input = parse_macro_input!(input as DeriveInput);
    let first_field = match &input.data {
        syn::Data::Struct(d) => d.fields.iter().next().unwrap(),
        _ => panic!("only structs supported"),
    };
    // ...
}

// ✅ 应该这样做
#[proc_macro_derive(Good)]
pub fn good_derive(input: TokenStream) -> TokenStream {
    let input = parse_macro_input!(input as DeriveInput);
    match good_derive_impl(&input) {
        Ok(tokens) => tokens.into(),
        Err(err) => err.to_compile_error().into(),
    }
}
```

`panic` 会导致:

- 没有准确的错误位置
- 无法看到其他错误 (编译直接终止)

- 用户不知道如何修复

9.8 小结

- 永远用 `syn::Error` 和 `?` 操作符——不要用 `unwrap()` 或 `panic!`
- `err.to_compile_error()` 把错误转化为 `compile_error!()` 宏
- `Span` 提供错误定位——`Error::new_spanned(value, msg)` 自动使用 `value` 的 `span`
- `cargo expand` 是调试的利器
- 单元测试用 `proc_macro2` 而非 `proc_macro`, 用 `parse_quote!` 快速构造输入
- 永远在顶层函数中使用 `Result` 模式, 而不是让 `panic` 泄漏出去

10 测试过程宏

 实验记录：第九站

过程宏在编译期运行，不能像普通代码那样直接写 `#[test]` 测试。但有几种行之有效的测试策略。

10.1 策略一：集成测试

在另一个 `crate` 中测试宏展开后的行为。过程宏 `crate` 可以有自己的 `tests/` 目录：

```
my-macro/  
├─ Cargo.toml  
├─ src/  
│   └─ lib.rs          # 宏定义  
└─ tests/  
    └─ integration.rs  # 集成测试
```

```
// tests/integration.rs  
use my_macro::MyTrait;  
use my_macro_derive::MyTrait;  
  
#[derive(MyTrait)]  
struct Point { x: i32, y: i32 }  
  
#[test]  
fn test_point_method() {  
    let p = Point { x: 1, y: 2 };  
    let result = p.my_method();  
    assert_eq!(result, "Point(1, 2)");  
}
```

概念

集成测试的优点：测试完整的宏展开流程，和用户使用方式一致。 缺点：需要在测试文件中定义类型，编译速度较慢。

10.2 策略二：用 `proc_macro2` 做单元测试

把核心逻辑和宏入口分离：

```
// src/lib.rs - 逻辑拆出去  
pub fn derive_my_trait_impl(  
    input: &syn::DeriveInput,  
) -> syn::Result<proc_macro2::TokenStream> {  
    let name = &input.ident;  
    // ... 实际逻辑  
    Ok(quote! { /* ... */ })  
}  
  
// 宏入口是薄薄的包装层  
#[proc_macro_derive(MyTrait)]  
pub fn derive_my_trait(input: TokenStream) -> TokenStream {  
    let input = parse_macro_input!(input as DeriveInput);  
    derive_my_trait_impl(&input)  
        .unwrap_or_else(|err| err.to_compile_error())  
}
```

```
        .into()
    }
}
```

单元测试：

```
#[cfg(test)]
mod tests {
    use super::*;
    use syn::parse_quote;

    #[test]
    fn test_basic_struct() {
        let input: syn::DeriveInput = parse_quote! {
            struct Point { x: i32, y: i32 }
        };
        let output = derive_my_trait_impl(&input).unwrap();
        assert!(output.to_string().contains("impl MyTrait for Point"));
    }

    #[test]
    fn test_generic_struct() {
        let input: syn::DeriveInput = parse_quote! {
            struct Wrapper<T: Display> { value: T }
        };
        let output = derive_my_trait_impl(&input).unwrap();
        let output_str = output.to_string();
        assert!(output_str.contains("impl<T: Display>"));
        assert!(output_str.contains("Wrapper<T>"));
    }

    #[test]
    fn test_enum_rejected() {
        let input: syn::DeriveInput = parse_quote! {
            enum MyEnum { A, B, C }
        };
        let result = derive_my_trait_impl(&input);
        assert!(result.is_err());
    }
}
```

核心直觉

`parse_quote!` 是 `syn` 提供的神奇宏——在测试中直接写 Rust 代码，自动解析成语法树节点。

```
let field: syn::Field = parse_quote! { name: String };
let ty: syn::Type = parse_quote! { Vec<u32> };
```

不需要从字符串解析，不需要关心 `TokenStream` 构造——直接写代码就行。

10.3 策略三：编译失败测试（`trybuild`）

测试宏在错误使用时是否给出友好的编译错误。

```
// tests/compile_fail.rs
#[test]
fn compile_fail_tests() {
    let t = trybuild::TestCases::new();
    t.compile_fail("tests/ui/*.rs");
}
```

在 `tests/ui/` 中放“应该编译失败”的文件：

```
// tests/ui/enum_not_supported.rs
use my_macro_derive::MyTrait;

#[derive(MyTrait)]
//~^ error: MyTrait can only be derived for structs
enum MyEnum { A, B }
```

`//~^` 注释告诉 `trybuild`：这个错误信息应该出现在上一行。

定义

`trybuild` 把源文件作为独立 `crate` 编译，检查编译结果是否符合预期。第一次运行会生成 `.stderr` 文件，之后每次测试都会比较输出是否一致。

10.4 策略四：快照测试

用 `insta crate` 验证生成的代码：

```
use insta;

#[test]
fn test_generated_code_snapshot() {
    let input: syn::DeriveInput = parse_quote! {
        struct Point { x: i32, y: i32 }
    };
    let output = derive_my_trait_impl(&input).unwrap();
    let output_str = prettyplease::unparse(
        &syn::parse_file(&output.to_string()).unwrap()
    );
    insta::assert_snapshot!("point_derive", output_str);
}
```

格式化的代码会被保存到快照文件中。修改宏后，`cargo insta review` 可以逐个审查变化。

10.5 测试策略总结

方法	适用场景	特点
集成测试	验证宏展开后的行为	最真实，但编译慢
单元测试 (<code>parse_quote!</code>)	验证逻辑正确性	快速，可精细控制
<code>trybuild</code>	验证错误信息质量	确保友好报错
<code>cargo expand</code>	手动检查展开结果	交互式调试
快照测试	防止回归	自动化对比

10.6 最佳实践

```
// tests/integration.rs - 功能验证
#[derive(MyTrait)]
struct Point { x: i32, y: i32 }
```

```

#[test]
fn it_works() {
    let p = Point { x: 1, y: 2 };
    assert_eq!(p.greeting(), "Hello from Point");
}

// tests/compile_fail.rs - 错误验证
#[test]
fn compile_fail() {
    let t = trybuild::TestCases::new();
    t.compile_fail("tests/ui/*.rs");
}

// src/lib.rs - 单元测试
#[cfg(test)]
mod tests {
    #[test]
    fn test_generics() { /* ... */ }
    #[test]
    fn test_empty_struct_rejected() { /* ... */ }
}

```

10.7 小结

- 拆分逻辑：核心逻辑放在独立函数中，宏入口只是包装层
- 使用 `parse_quote!` 在测试中快速构造 `syn` 节点
- 使用 `trybuild` 测试编译失败场景
- 使用快照测试（`insta`）防止代码生成回归
- `cargo expand` 是方便的交互式调试工具
- 好的测试方案 = 单元测试 + 集成测试 + 编译失败测试

11 实战：从零实现 `derive(Builder)`

 实验记录：第十站

现在做一个完整的东西——一个 `#[derive(Builder)]` 宏，为结构体自动生成 `Builder` 模式。

这是 Rust 生态中最经典的 `derive` 宏用例之一（`derive_builder`、`typed-builder` 等 crate 都在做类似的事）。

11.1 目标

用户写：

```
#[derive(Builder)]
struct Command {
    executable: String,
    #[builder(default)]
    args: Vec<String>,
    #[builder(default = "vec![ ]")]
    env: Vec<String>,
    current_dir: Option<String>,
}
```

然后能用：

```
let cmd = Command::builder()
    .executable("cargo".to_string())
    .args(vec!["build".to_string()])
    .build()
    .unwrap();
```

11.2 项目结构

```
cargo new builder-derive --lib
```

Cargo.toml:

```
[lib]
proc-macro = true

[dependencies]
syn = { version = "2.0", features = ["full"] }
quote = "1.0"
```

11.3 核心设计

这是写过程宏的关键设计模式：

1. 遍历字段，为每个字段收集信息
2. 为每个字段生成一个“builder 版本”（`Option<T>`）
3. 为每个字段生成一个 `setter` 方法
4. 在 `build()` 中检查必填字段，用 `unwrap_or_default()` 处理可选字段

11.4 解析阶段

```
use proc_macro::TokenStream;
use syn::{parse_macro_input, DeriveInput, Data, Fields};
use quote::quote;

#[proc_macro_derive(Builder, attributes(builder))]
pub fn derive_builder(input: TokenStream) -> TokenStream {
    let input = parse_macro_input!(input as DeriveInput);

    match derive_builder_impl(&input) {
        Ok(tokens) => tokens.into(),
        Err(err) => err.to_compile_error().into(),
    }
}

fn derive_builder_impl(input: &DeriveInput)
-> syn::Result<proc_macro2::TokenStream>
{
    let struct_name = &input.ident;
    let builder_name = syn::Ident::new(
        &format!("{}", Builder, struct_name),
        struct_name.span(),
    );

    let fields = match &input.data {
        Data::Struct(data) => &data.fields,
        _ => return Err(syn::Error::new_spanned(
            input, "Builder can only be derived for structs")),
    };

    let (impl_generics, ty_generics, where_clause) =
        input.generics.split_for_impl();

    // 遍历字段，收集信息
    let mut builder_fields = Vec::new();
    let mut setter_methods = Vec::new();
    let mut initializers = Vec::new();

    for field in fields.iter() {
        let field_name = field.ident.as_ref()
            .ok_or_else(|| syn::Error::new_spanned(
                field, "tuple structs not supported"))?;
        let field_type = &field.ty;

        let has_default = field.attrs.iter().any(|attr| {
            attr.path().is_ident("builder")
        });

        // Builder 结构体字段: Option<FieldType>
        builder_fields.push(quote! {
            #field_name: Option<#field_type>
        });

        // setter 方法
        setter_methods.push(quote! {
```

```

        pub fn #field_name(mut self, value: #field_type) -> Self {
            self.#field_name = Some(value);
            self
        }
    });

    // build() 中的初始化
    if has_default {
        initializers.push(quote! {
            #field_name: self.#field_name
                .unwrap_or_default()
        });
    } else {
        initializers.push(quote! {
            #field_name: self.#field_name
                .ok_or_else(|| format!(
                    "field `{}` is required",
                    stringify!(#field_name)
                ))?
        });
    }
}

// 生成代码
let expanded = generate_builder_code(
    struct_name, builder_name,
    impl_generics, ty_generics, where_clause,
    &builder_fields, &setter_methods, &initializers,
);

Ok(expanded)
}

```

11.5 代码生成阶段

```

fn generate_builder_code(
    struct_name: &syn::Ident,
    builder_name: syn::Ident,
    impl_generics: impl quote::ToTokens,
    ty_generics: impl quote::ToTokens,
    where_clause: impl quote::ToTokens,
    builder_fields: &[proc_macro2::TokenStream],
    setter_methods: &[proc_macro2::TokenStream],
    initializers: &[proc_macro2::TokenStream],
) -> proc_macro2::TokenStream {
    quote! {
        // Builder 结构体
        pub struct #builder_name #ty_generics #where_clause {
            #(#builder_fields),*
        }

        // Builder 的 new() 和 setter 方法
        impl #impl_generics #builder_name #ty_generics #where_clause {
            pub fn new() -> Self {
                Self {

```

```

        #( #builder_fields: None ),*
    }
}

#(#setter_methods)*

pub fn build(self) -> Result<#struct_name #ty_generics, String> {
    Ok(#struct_name {
        #( #initializers ),*
    })
}

// 为原结构体添加 builder() 方法
impl #impl_generics #struct_name #ty_generics #where_clause {
    pub fn builder() -> #builder_name #ty_generics {
        #builder_name::new()
    }
}
}
}

```

11.6 完整的展开示例

输入:

```

#[derive(Builder)]
struct Command {
    executable: String,
    #[builder(default)]
    args: Vec<String>,
}

```

展开为:

```

pub struct CommandBuilder {
    executable: Option<String>,
    args: Option<Vec<String>>,
}

impl CommandBuilder {
    pub fn new() -> Self {
        Self { executable: None, args: None }
    }

    pub fn executable(mut self, value: String) -> Self {
        self.executable = Some(value);
        self
    }

    pub fn args(mut self, value: Vec<String>) -> Self {
        self.args = Some(value);
        self
    }
}

```

```

pub fn build(self) -> Result<Command, String> {
    Ok(Command {
        executable: self.executable
            .ok_or_else(|| "field `executable` is required".to_string()),
        args: self.args.unwrap_or_default(),
    })
}

impl Command {
    pub fn builder() -> CommandBuilder { CommandBuilder::new() }
}

```

11.7 进阶改进

11.7.1 1. 支持 #[builder(default = "expr")]

```

fn get_default_value(field: &syn::Field) -> Option<proc_macro2::TokenStream> {
    for attr in &field.attrs {
        if attr.path().is_ident("builder") {
            let mut default = None;
            attr.parse_nested_meta(|meta| {
                if meta.path.is_ident("default") {
                    let value = meta.value()?;
                    let expr: syn::Expr = value.parse()?;
                    default = Some(quote! { #expr });
                }
                Ok(())
            }).ok()?;
            return default;
        }
    }
    None
}

```

11.7.2 2. 支持文档注释转发

```

let doc_attrs: Vec<_> = field.attrs.iter()
    .filter(|a| a.path().is_ident("doc"))
    .collect();

quote! {
    #(#doc_attrs)*
    pub fn #field_name(mut self, value: #field_type) -> Self { ... }
}

```

11.7.3 3. 一次报告所有缺失字段

```

let check_required: Vec<_> = fields.iter().filter_map(|field| {
    let has_default = field_has_default(field);
    if has_default { return None; }
    let field_name = &field.ident;
    Some(quote! {

```

```

        if self.#field_name.is_none() {
            missing.push(stringify!(#field_name));
        }
    })
}).collect();

quote! {
    pub fn build(self) -> Result<#struct_name, Vec<String>> {
        let mut missing = Vec::new();
        #(#check_required)*
        if !missing.is_empty() { return Err(missing); }
        Ok(#struct_name { #(#initializers),* })
    }
}

```

11.8 常见陷阱

⚠ 注意

Builder 宏的几个陷阱:

1. **生命周期**: 如果字段包含引用, **builder** 需要处理生命周期参数
2. **默认值和必填字段的区分**: `Option` 本身可以是合法的字段值
3. **错误收集**: 一次报告所有缺失字段, 而不是第一个就终止
4. **Builder 的 consuming vs borrowing**: 这里用了 `mut self` (消费), 也可以用 `&mut self` (借用)

11.9 小结

- **Builder** 宏架构: 遍历字段 → 生成 **builder** 结构体 → 生成 **setter** → 生成 **build()**
- 用 `#[builder(default)]` 标记可选字段
- 用 `ok_or_else` 在 **build()** 中检查必填字段
- 用 `format!("{}", ...)` 生成友好的缺失字段错误信息
- `parse_nested_meta` 处理属性参数
- 考虑生命周期、错误聚合等进阶需求

12 进阶主题

🔧 实验记录： 第十一站

至此我们已经掌握了过程宏的基础流程。现在来看一些更深层的机制。

12.1 卫生性 (Hygiene)

概念

词法卫生性 (lexical hygiene) 是宏系统最重要的保证之一——防止宏内部生成的标识符意外地和用户代码冲突。

在 `macro_rules!` 中这是自动的。在过程宏中.....需要手动控制。

```
// 宏生成的代码
let temp = 42;

// 如果用户代码也有 temp:
let temp = "hello";
my_macro!(); // 这里会冲突吗？
```

过程宏中生成的 `temp` 默认使用调用位置的 `span`——它会和外部在同一个作用域。

解决方案：使用 唯一标识符：

```
use quote::format_ident;

// 为每个宏调用生成唯一的标识符
let uid: u64 = /* 生成唯一 ID */;
let temp_var = format_ident!("__temp_{}", uid, span = span);

quote! {
    let #temp_var = 42;
    // 使用 #temp_var
}
```

12.2 Span 的深层语义

概念

`Span` 不只是“错误定位”——它还参与 `Rust` 的隐私检查和卫生性。

- `Span::call_site()` – 使用宏调用位置的 `span`。此标识符可以和外部代码交互。
- `Span::def_site()` – 使用宏定义位置的 `span`。此标识符是“完全卫生”的，外部无法访问。
- 自定义 `Span` – 如果你知道自己在做什么，可以直接控制 `span`。

```
#[proc_macro]
pub fn make_answer(input: TokenStream) -> TokenStream {
    // ❌ def_site 生成的标识符无法访问用户 crate 中的类型
    let helper = Ident::new("Helper", Span::def_site());

    // ✅ call_site 生成的标识符可以访问宏调用位置
    let helper = Ident::new("Helper", Span::call_site());

    quote! {
```

```

        struct #helper;
        impl MyTrait for #helper { ... }
    }
}

```

不正确的 `span` 选择会导致“找不到类型”或“`trait` 未实现”等奇怪错误。

12.3 模块化

当宏变得复杂时，需要拆分文件：

```

// lib.rs
mod parsing;
mod codegen;
mod attrs;

#[proc_macro_derive(Builder, attributes(builder))]
pub fn derive_builder(input: TokenStream) -> TokenStream {
    let input = parse_macro_input!(input as DeriveInput);
    builder_impl::expand(input)
        .unwrap_or_else(|err| err.to_compile_error())
        .into()
}

```

```

// parsing.rs - TokenStream -> 配置数据结构
pub(super) fn parse_builder_attrs(field: &syn::Field) -> BuilderFieldConfig
{ ... }

// codegen.rs - 配置 -> TokenStream
pub(super) fn generate_builder_struct(config: &BuilderConfig) ->
proc_macro2::TokenStream { ... }

// attrs.rs - #[builder(...)] 属性解析
pub(super) struct BuilderFieldConfig {
    pub has_default: bool,
    pub default_expr: Option<proc_macro2::TokenStream>,
}

```



概念

模块化原则：

- 解析逻辑：把 `TokenStream` 变成配置
- 代码生成：把配置变成 `TokenStream`
- 入口：薄薄的包装层

12.4 编译时资源嵌入

过程宏可以在编译时读取文件、解析内容，生成对应的 `Rust` 代码：

```

#[proc_macro]
pub fn include_sql(input: TokenStream) -> TokenStream {
    let file_path: syn::LitStr = parse_macro_input!(input as syn::LitStr);
    let path = file_path.value();
}

```

```

// 编译时读取 SQL 文件
let sql = std::fs::read_to_string(&path)
    .expect("failed to read SQL file");

// 编译时解析 SQL
let parsed = parse_sql(&sql);

// 生成类型安全的 Rust 代码
let expanded = generate_typesafe_query(parsed);
expanded.into()
}

```

这就是 `sqlx::query!` 的原理——编译时读取 SQL 文件，解析它，生成对应类型的行结构体。

12.5 发布过程宏 `crate`

通常过程宏 `crate` 分两个包：

```

my_macro/           # 用户依赖的包，导出 trait 和重新导出宏
my_macro_derive/    # 实际的 proc-macro crate（用户不直接依赖）

```

```

# my_macro/Cargo.toml
[dependencies]
my_macro_derive = { version = "0.1", path = "../my_macro_derive" }

```

```

// my_macro/src/lib.rs
pub use my_macro_derive::MyMacro;
pub trait MyMacro { ... }

```

依赖注意事项：

```

[dependencies]
syn = { version = "2.0", features = ["derive"] } # 够用就行
quote = "1.0"
proc-macro2 = "1.0"

```

- `syn` 用 `features = ["derive"]` 即可（不需要 `full` 就尽量不加）
- 用最新稳定版，不依赖 `nightly`
- 不要引入运行时依赖

12.6 过程宏的限制

⚠ 注意

1. 只能生成代码 — 不能检查类型信息（类型检查在宏展开之后）
2. 不能有副作用 — 除了读取文件，不能访问网络等
3. 不能跨越模块边界 — 一个宏不能直接修改另一个模块的代码
4. 编译时间 — 复杂宏会显著增加编译时间
5. 递归限制 — 宏展开有深度限制
6. 不支持跨 `crate` 的宏状态 — 每个 `crate` 独立编译宏

12.7 未来方向

| 特性 | 状态 | 说明 | |:-|:-|:-| | `proc_macro_diagnostic` | `nightly` | 更丰富的诊断信息 | | `proc_macro_span` | `nightly` | 更精确的 `span` 操作 | | `proc_macro_tracked_env` | `nightly` | 跟踪环境变量变化 | | 类型化宏参数 | RFC | 更安全地传递类型信息 |

即使没有 `nightly` 特性，稳定版的过程宏已经足够强大——`serde`、`tokio`、`sqlx`、`axum` 等核心生态都构建在它们之上。

12.8 本质

过程宏表面上是代码生成，但本质是 **编译时计算**。你把一部分决策工作从运行时挪到编译时——生成代码的逻辑本身在编译期运行，生成的代码在运行时执行。这是一个双层的计算模型。

理解了这一点，过程宏就不只是“省力工具”——它是一种强大的程序变换方式。

12.9 小结

- 卫生性：用 `Span` 控制标识符的可见范围，用 `format_ident!` 创建唯一变量名
- 模块化：解析逻辑、代码生成、属性处理分离
- 编译时资源嵌入：读取文件、解析 SQL、生成类型安全代码
- 发布策略：分离为 `proc-macro crate` 和 `facade crate`
- 限制：不能检查类型信息、有编译开销
- 本质：编译时计算——把决策从运行时提前到编译时

13 回顾与展望

🔗 实验记录： 终点站

13.1 回顾旅程

从重复代码的困扰，到一个完整的 `#[derive(Builder)]` 宏——我们走过了很长的路。

| 概念 | 关键内容 | |:-|:-| | `TokenStream` | 编译器表示代码的内部结构，四种 `TokenTree`
| | 三种过程宏 | 函数式宏、派生宏、属性宏 | | `syn` 解析 | 把 `TokenStream` 解析成
`DeriveInput` 等数据结构 | | `quote` 生成 | 安全地生成 `TokenStream` | | `Derive Macro`
| 为类型自动实现 `trait` | | `Attribute Macro` | 注解函数/结构体，包裹或转换代码 | |
`Function-like Macro` | 创造自定义 DSL | | `syn::Error` | 给用户友好的编译错误 | |
`trybuild` | 测试编译失败场景 | | `Builder` 实战 | 完整的 `derive` 宏实现 | | 卫生性与
`Span` | 控制标识符的作用域规则 |

13.2 概念速查表

| 类型 | 标注 | 用途 | |:-|:-|:-| | `#[proc_macro]` | 函数式宏 | 自定义 DSL 语法 `my_macro!`
(...) | | `#[proc_macro_derive]` | 派生宏 | 为类型自动实现 `trait` `#[derive(Trait)]` | |
`#[proc_macro_attribute]` | 属性宏 | 注解函数/结构体 `#[attr]` | | `proc_macro::TokenStream`
| 内置类型 | 编译器传入传出的 `token` 序列 | | `proc_macro2::TokenStream` | 跨平台封装
| 可在普通代码和测试中使用 | | `syn::DeriveInput` | 解析结构 | `derive` 宏的输入数据结构
| | `syn::ItemFn` | 解析结构 | 函数定义的语法树 | | `quote!` | 代码生成 | 安全地生成
`TokenStream` | | `syn::Error` | 错误处理 | 生成友好的编译错误 | | `parse_quote!` | 测试
辅助 | 在测试中快速构造 `syn` 节点 | | `trybuild` | 测试框架 | 测试编译失败场景 |

13.3 推荐阅读

进阶资料：

- **The Little Book of Rust Macros** – `macro_rules!` 的全面教程
- **syn 文档** – 所有解析类型的详细说明
- **quote 文档** – 插值模式的高级用法
- **serde 源码** – 最经典的 `derive` 宏实现
- **tokio 的 `#[tokio::main]`** – 属性宏的优雅应用
- **sqlx 的 `query!`** – 函数式宏 + 编译时 SQL 解析
- **Rust Reference: Procedural Macros** – 官方规范的权威描述

13.4 最后的话

过程宏让 Rust 具备了语言最强大的元编程能力。但强大意味着责任——好的宏不是“能工作就行”，而是“让使用它的人感觉不到宏的存在”。

给宏作者的几条建议：

1. 用户友好的错误信息 – 不要 `panic`，不要 `unwrap`
2. 正确处理泛型 – 你的宏必须工作在泛型代码中
3. 测试 – 单元测试 + 集成测试 + 编译失败测试
4. 文档 – 写清楚宏做什么、怎么用、错误意味着什么
5. 模块化 – 复杂宏要拆分文件，逻辑清晰

过程宏的本质是 **编译时计算**——你把一部分逻辑从运行时提前到编译时执行。这不仅减少了重复代码，还让生成的代码拥有手工编写级别的性能。

在 Rust 的世界里，只要你能 `parse` 它，你就能生成它。

